VitaStress — Passi 1 e 2: tabella e classificatore

Costruzione dei dati e classificatore binario stress / non stress · Tirocinio, Interfacce uomo-macchina

Parte 1 — La tabella

1. A cosa serve questo passo

Un classificatore non sa leggere registrazioni lunghe un'ora e mezza: vuole una **tabella**, in cui ogni riga è un esempio e una colonna dice la risposta giusta. Questo passo costruisce quella tabella.

La costruiamo **come l'hanno costruita gli autori**, con gli stessi dati e la stessa durata dei pezzi. Serve come **punto di riferimento**: prima rifacciamo il loro esperimento e otteniamo il nostro numero; poi, al passo 4, cambieremo solo le feature (usando i dati grezzi) e vedremo quanto migliorano. Se tutto il resto resta uguale, la differenza è merito delle feature.

> **In una frase:** trasformare le registrazioni di 21 persone in una tabella con una riga ogni 30 secondi, le misure di quei 30 secondi e l'etichetta stress / non stress.

2. Cosa hanno fatto gli autori

Dalla sezione 3.6 del paper arXiv (2508.10468):

- usano i **dati già elaborati dal braccialetto** per battito, sudore e temperatura
- dall'**accelerometro grezzo** calcolano media, deviazione standard e varianza di X, Y, Z
- portano tutto a **un valore ogni 30 secondi**
- **normalizzano ogni persona** rispetto a sé stessa
- per il binario: **riposo = non stress**, **calcolo + discorso + corsa = stress**

Il dettaglio è nel documento `docs/feature_autori.md`.

3. Com'è fatta la tabella

Ogni riga è un pezzo da 30 secondi di una persona. Le colonne sono di tre tipi:

colonne	ruolo
bpm, stress_skin, temp_sk1, accX/Y/Z_mean, _std, _var	**le feature**: quello che il modello guarda (12 colonne)
stress	**l'etichetta**: quello che il modello deve indovinare (0 o 1)
persona, fase, date	servono per dividere i dati e per controllare; il modello non le vede

Quattro righe vere della persona 840e79d3, una per fase, con i valori originali (solo alcune colonne; «sudore» è stress_skin, «temp.» è temp_sk1):

fase	bpm	sudore	temp.	accX media	accX std	accX var	stress
baseline	80	12	37,1	−497	9	84	0
cognitive	111	241	37,3	−394	6	32	1
public	104	7	37,1	−363	18	316	1
physical	82	4	37,2	−131	76	5.765	1

Si vede già qualcosa: nel calcolo battito e sudore sono molto più alti; nella corsa la deviazione standard e la varianza dell'accelerometro esplodono, perché il braccio si muove molto.

4. Il codice, fase per fase

Fase 0 · Setup

```
import os
import pandas as pd
import numpy as np

if not os.path.exists("VitaStress"):
    !git clone https://github.com/paulvincenz/VitaStress.git

DATA = "VitaStress/data"
persone = sorted([c.replace("id_", "") for c in os.listdir(DATA) if c.startswith("id_")])
```

Librerie, download del dataset se manca, elenco dei 21 soggetti. È un notebook nuovo: non conosce nulla di quello della Figura 4, quindi va tutto riscritto.

Fase 1 · Orari delle fasi

```
def trova_orari(persona):
    ...   # identica al notebook della Figura 4
```

La stessa funzione `trova_orari` della Figura 4: dalle annotazioni ricava inizio e fine di ogni fase, salta quelle mancanti o con durata impossibile.

Fase 2 · Segnali ogni 30 secondi

```
def leggi(cartella, file, colonna):
    dati = pd.read_csv(cartella + file + ".csv", usecols=["date", colonna])
    dati["date"] = pd.to_datetime(dati["date"], format="mixed")
    return dati.set_index("date")[colonna]

def segnali_30s(persona):
    cartella = DATA + "/id_" + persona + "/" + persona + "_"

    bpm = leggi(cartella, "activity", "bpm").replace(0, np.nan)
    temp = leggi(cartella, "temperature", "temp_sk1").replace(0, np.nan)
    sudore = leggi(cartella, "emography", "stress_skin")

    tab = pd.DataFrame({
        "bpm": bpm.resample("30s").mean(),
        "temp_sk1": temp.resample("30s").mean(),
        "stress_skin": sudore.resample("30s").mean().ffill(limit=1),
    })
    for asse in ["accX", "accY", "accZ"]:
        acc = leggi(cartella, "acc", asse).resample("30s")
        tab[asse + "_mean"] = acc.mean()
        tab[asse + "_std"] = acc.std()
        tab[asse + "_var"] = acc.var()
    return tab
```

- `leggi` apre un file e restituisce una colonna, con l'orario come indice (la «chiave» di ogni riga)
- `resample("30s")` divide il tempo in **blocchi da 30 secondi**; `.mean()` fa la media di ogni blocco
- battito e temperatura a 0 diventano mancanti (errore del sensore), come nella Figura 4
- **sudore**: il braccialetto dà un valore ogni 60 secondi. Dividendo in blocchi da 30, un blocco sì e uno no resterebbe vuoto: `ffill(limit=1)` lo riempie con il valore precedente, al massimo per un blocco
- **accelerometro**: 32 valori al secondo, quindi circa 960 per blocco. Per ogni blocco calcolo **media** (dove sta il polso), **deviazione standard** e **varianza** (quanto si muove), come gli autori
- risultato: una tabella con una riga ogni 30 secondi per tutta la registrazione della persona

Fase 3 · Fasi ed etichetta

```
def tabella_persona(persona):
    tab = segnali_30s(persona)
    orari = trova_orari(persona)
    pezzi = []
    for fase in orari:
        inizio, fine = orari[fase]
        pezzo = tab[(tab.index >= inizio) & (tab.index < fine)].copy()
        pezzo["fase"] = fase
        pezzi.append(pezzo)
    tab = pd.concat(pezzi)
    tab["persona"] = persona
    tab["stress"] = (tab["fase"] != "baseline").astype(int)
    return tab
```

- tiene solo i blocchi che cadono dentro una fase, e ci scrive la fase: è il `taglia` della Figura 4. Pause, questionari e preparazioni restano fuori
- aggiunge il codice della persona: servirà per dividere i dati al passo 2

- **l'etichetta**: `fase != "baseline"` è vero per calcolo, discorso e corsa; `astype(int)` trasforma vero in **1** e falso in **0**

Fase 4 · Tutti i soggetti

```
tabella = pd.concat([tabella_persona(p) for p in persone]).reset_index()
```

Chiama `tabella_persona` per ognuno dei 21 soggetti e unisce i risultati. `reset_index` trasforma l'orario da indice a colonna normale (`date`). Ci mette circa 30 secondi. Qui la tabella è **costruita ma non ancora pronta**: ha ancora i buchi e i valori originali.

Fase 5 · Dati mancanti

```
colonne = ["bpm", "stress_skin", "temp_sk1",
           "accX_mean", "accX_std", "accX_var", "accY_mean", "accY_std", "accY_var",
           "accZ_mean", "accZ_std", "accZ_var"]

mancanti = tabella.groupby("fase")[colonne].apply(lambda t: t.isna().mean() * 100).round(1)
```

- `colonne` è l'elenco delle 12 feature
- per ogni fase e colonna, `isna().mean() * 100` dà la **percentuale di valori mancanti**

Fase 6 · Pulizia e normalizzazione

```
tabella = tabella.dropna(subset=colonne)

for c in colonne:
    tabella[c] = tabella.groupby("persona")[c].transform(lambda s: (s - s.mean()) / s.std())

tabella.to_csv("tabella_30s.csv", index=False)
```

- `dropna` toglie le righe con almeno una feature mancante
- la **normalizzazione per persona** è la stessa della Figura 4: `groupby("persona")` calcola media e deviazione standard di ogni persona separatamente, e `transform` rimette i valori al loro posto nella tabella
- `to_csv` salva la tabella pronta, così al passo 2 non serve ricalcolarla

5. Il risultato

	righe
dopo la Fase 4 (tabella costruita)	1.170
tolte per dati mancanti	25
dopo la Fase 6 (tabella pronta)	**1.145**

Righe per fase e per classe

fase	righe (Fase 4)	righe (Fase 6)	etichetta
baseline	403	403	0 — non stress
cognitive	201	195	1 — stress
public	215	205	1 — stress
physical	351	342	1 — stress
totale	**1.170**	**1.145**	**403 × 0, 742 × 1**

Il riposo ha 403 righe, cioè 20 persone × circa 20 blocchi: manca la persona a5e823ad, il cui riposo da 82 minuti viene escluso.

Dati mancanti (percentuale, prima della pulizia)

colonna	baseline	cognitive	physical	public
bpm	0,0	3,0	2,3	4,7
temp_sk1	0,0	0,5	0,3	0,0
stress_skin	0,0	0,0	0,0	0,0
accelerometro (tutte)	0,0	0,0	0,0	0,0

Mancano pochissimi dati: solo il battito, un po' di più nelle fasi di stress. Sono 25 righe su 1.170, quindi toglierle è la scelta più semplice e non cambia la composizione della tabella.

La tabella pronta

Le stesse quattro righe di prima dopo la normalizzazione per persona: 0 è il valore abituale della persona, sopra o sotto indica quanto se ne allontana.

fase	bpm	sudore	temp.	accX media	accX std	accX var	stress
baseline	−0,62	−0,19	−0,71	−1,46	−0,39	−0,28	0
cognitive	1,05	4,35	1,57	−0,68	−0,41	−0,29	1
public	0,68	−0,29	−0,85	−0,45	−0,33	−0,28	1
physical	−0,51	−0,35	0,00	1,30	0,04	−0,23	1

6. Due cose da tenere a mente

Le classi non sono bilanciate. Ci sono 742 righe di stress e 403 di riposo, circa due terzi contro un terzo. Un modello che risponde sempre «stress» avrebbe già il **65%** di accuratezza senza aver imparato niente. Quando confronteremo con l'89% del paper, il punto di partenza non è il 50%. Per questo guarderemo anche l'F1-score, che risente meno dello sbilanciamento.

I dati mancanti non devono diventare un indizio. Il battito manca un po' più spesso nelle fasi di stress. Se la tabella avesse una colonna «battito mancante sì/no», il modello potrebbe usarla per indovinare la fase senza capire nulla dello stress. Per questo le righe incomplete vengono tolte e non si aggiunge mai una colonna del genere.

7. Decisioni prese

#	decisione	motivo
1	un valore ogni 30 secondi	come gli autori
2	accelerometro: media, dev. standard, varianza per blocco	come gli autori
3	battito 0 e temperatura 0 °C → mancanti	errori del sensore
4	sudore da 60 s a 30 s riempiendo con il valore precedente	stesso ritmo delle altre misure
5	righe con dati mancanti tolte (25 su 1.170)	pochi casi; evita che il mancante diventi un indizio
6	normalizzazione per persona	come gli autori
7	binario: riposo = 0; calcolo, discorso, corsa = 1	come il paper (Tabella 4)
8	a5e823ad: riposo escluso	82 minuti, pulsante premuto male

Parte 2 — Il classificatore binario

8. L'idea

Un classificatore impara da esempi già etichettati. Gli si mostrano tante righe della tabella insieme alla risposta giusta (stress o non stress); lui cerca una regola che colleghi i numeri alla risposta. Poi gli si mostrano righe nuove, senza risposta, e si conta quante ne indovina.

nel codice	colonne	ruolo
X	le 12 feature	cosa il modello guarda
y	stress (0 o 1)	cosa deve indovinare
gruppi	persona	per lasciare fuori una persona alla volta

9. La validazione leave-one-subject-out

Per misurare se il modello funziona davvero bisogna provarlo su dati che **non ha mai visto**. Qui «non visto» vuol dire **una persona che non ha mai visto**:

giro	addestramento	prova
1	persone 2–21	persona 1
2	persone 1, 3–21	persona 2
…	…	…
21	persone 1–20	persona 21

Alla fine si hanno 21 risultati, uno per persona: se ne fa la **media** e la **deviazione standard**. La media dice quanto funziona in generale; la deviazione standard quanto cambia da una persona all'altra.

Perché non dividere a caso, 70/30 come all'esame? Mescolando le righe, pezzi della stessa persona finirebbero sia nell'addestramento sia nella prova. Il modello potrebbe riconoscere la persona invece dello stress, e l'accuratezza sarebbe gonfiata. È anche la validazione usata dagli autori, quindi i numeri sono confrontabili.

10. I sei classificatori

Sono gli stessi del paper. In breve, come ragiona ciascuno:

sigla	nome	come ragiona
LR	Logistic Regression	traccia un confine «dritto» fra le due classi, pesando ogni feature
LDA	Linear Discriminant Analysis	anche lui un confine dritto, cercando la direzione che separa meglio le classi
DT	Decision Tree	una serie di domande sì/no (es. «battito > 0,5?») fino alla risposta
RF	Random Forest	tanti alberi di decisione, ognuno su una parte dei dati; vince la maggioranza
XGB	XGBoost	alberi costruiti uno dopo l'altro, ognuno corregge gli errori dei precedenti
kNN	k-Nearest Neighbours	guarda le 5 righe più simili già viste e copia la risposta più frequente

11. Le metriche

- **accuracy**: la percentuale di righe indovinate
- **F1-score**: tiene conto sia degli stress mancati sia dei falsi allarmi. Qui è calcolato per ciascuna classe e poi mediato (*macro*), come nel paper. È più affidabile dell'accuracy quando le classi sono sbilanciate
- **termine di paragone**: rispondendo sempre «stress» si otterrebbe il **65%** di accuracy senza imparare nulla

12. Il codice, fase per fase

Fase 7 · Feature ed etichetta

```
X = tabella[colonne]
y = tabella["stress"]
gruppi = tabella["persona"]

print("Righe:", len(X), "| feature:", X.shape[1])
print("Rispondendo sempre 'stress':", round(y.mean() * 100, 1), "%")
```

- separa la tabella nelle tre parti: feature, etichetta, persona
- `y.mean()` è la percentuale di righe con stress = 1: è l'accuracy del modello che risponde sempre «stress»

Fase 8 · I sei classificatori

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier

modelli = {
    "LR": LogisticRegression(max_iter=1000),
    "RF": RandomForestClassifier(n_estimators=100, random_state=42),
    "DT": DecisionTreeClassifier(random_state=42),
    "LDA": LinearDiscriminantAnalysis(),
    "XGB": XGBClassifier(random_state=42),
    "kNN": KNeighborsClassifier(n_neighbors=5),
}
```

- importa i sei modelli e li mette in un dizionario: nome → modello
- `random_state=42` fissa la casualità: rieseguendo si ottengono sempre gli stessi numeri
- `n_estimators=100`: 100 alberi nella foresta; `n_neighbors=5`: 5 vicini per il kNN; `max_iter=1000`: più tentativi alla Logistic Regression per trovare la soluzione
- sono impostazioni standard: il paper usa una grid search per sceglierle, ma non dice quali valori ha provato

Fase 9 · Validazione leave-one-subject-out

```
from sklearn.model_selection import LeaveOneGroupOut
from sklearn.metrics import accuracy_score, f1_score

def valuta(modello, X, y, gruppi):
    acc, f1 = [], []
    for train, test in LeaveOneGroupOut().split(X, y, gruppi):
        modello.fit(X.iloc[train], y.iloc[train])
        previsto = modello.predict(X.iloc[test])
        acc.append(accuracy_score(y.iloc[test], previsto))
        f1.append(f1_score(y.iloc[test], previsto, average="macro"))
    return np.mean(acc), np.std(acc), np.mean(f1), np.std(f1)
```

- `LeaveOneGroupOut().split(...)` genera i 21 giri: ogni volta dà le righe di addestramento (`train`) e quelle della persona esclusa (`test`)
- `fit` addestra il modello sulle 20 persone; `predict` gli fa indovinare la 21ª
- per ogni giro salva accuracy e F1; alla fine restituisce media e deviazione standard

Fase 10 · Risultati

```
risultati = []
for nome in modelli:
    a, a_sd, f, f_sd = valuta(modelli[nome], X, y, gruppi)
    risultati.append({"modello": nome,
                      "accuracy": f"{a:.2f} ± {a_sd:.2f}",
                      "F1": f"{f:.2f} ± {f_sd:.2f}"})

pd.DataFrame(risultati)
```

Valuta i sei modelli uno dopo l'altro e mette i risultati in una tabella. `f"{a:.2f}"` scrive il numero con due decimali.

13. I risultati

modello	accuracy (noi)	accuracy (paper, Tab. 4)	F1 (noi)
LR	0,92 ± 0,13	0,88 ± 0,13	0,90 ± 0,17
RF	**0,93 ± 0,11**	**0,89 ± 0,11**	0,91 ± 0,16
DT	0,91 ± 0,09	0,88 ± 0,13	0,88 ± 0,15
LDA	0,91 ± 0,14	0,88 ± 0,10	0,89 ± 0,18
XGB	0,94 ± 0,10	0,87 ± 0,14	0,91 ± 0,16
kNN	0,87 ± 0,16	0,87 ± 0,13	0,84 ± 0,21
sempre «stress»	*0,65*	—	—

14. Come leggerli

- **Il paper è riprodotto, anzi siamo un po' sopra**: 0,87–0,94 contro 0,87–0,89. Random Forest è fra i migliori, come per gli autori
- **i modelli hanno imparato qualcosa**: sono molto sopra il 65% di chi risponde sempre «stress»
- **la deviazione standard è grande** (± 0,10–0,16): il modello va molto bene su alcune persone e peggio su altre. Il paper nota la stessa cosa
- **perché qualche punto sopra?** Non si sa con certezza. Differenze possibili: abbiamo tolto battito e temperatura a 0, teniamo solo i blocchi interni alle fasi, non sappiamo come loro abbiano trattato i dati mancanti né quali iperparametri abbiano scelto. Va annotato come differenza da spiegare, non come miglioramento

Una nota di metodo. La normalizzazione per persona usa media e deviazione standard calcolate su tutte le righe della persona, compresa quella di prova. Non usa le etichette, ed è la stessa scelta degli autori, ma va dichiarata: in un uso reale servirebbe un tratto di registrazione a riposo per «tarare» ogni nuova persona.

15. Anteprima del passo 3

Random Forest con tre gruppi di feature diversi:

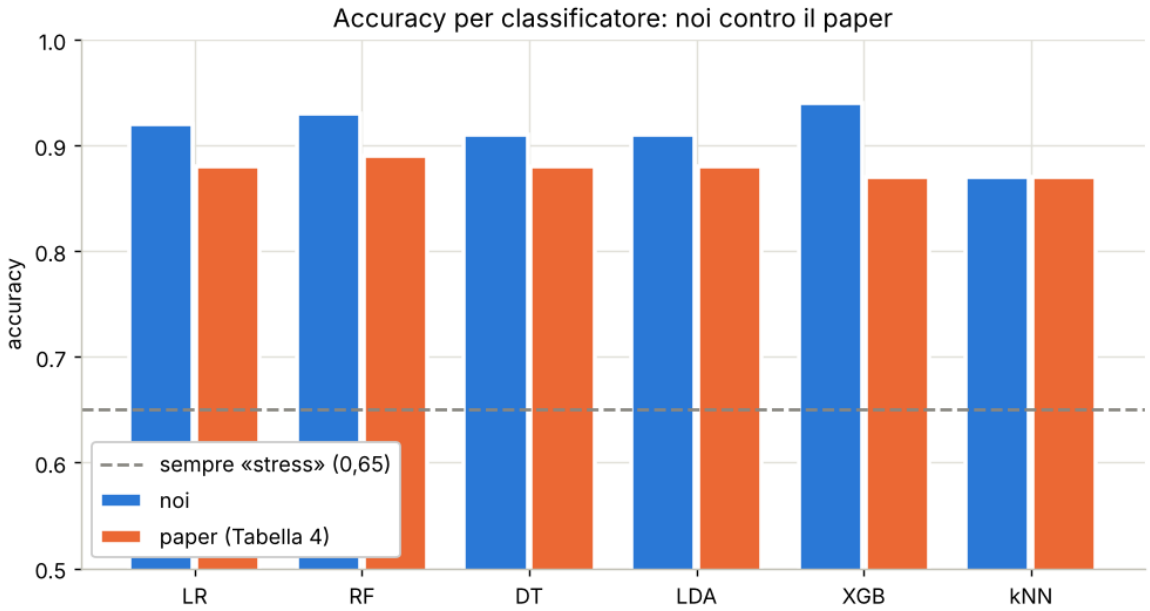
dati	feature	accuracy
tutti	12	0,93
solo fisiologia	bpm, stress_skin, temp_sk1	0,90
solo movimento	**le 9 dell'accelerometro**	**0,92**

Il solo movimento fa quasi quanto tutti i segnali insieme, come nel paper. È il punto da cui parte il progetto: il passo 3 lo approfondisce, anche togliendo la corsa.

Parte 3 — I grafici

Quattro grafici per leggere i risultati. Per ognuno: cosa c'è nel grafico, come si legge e cosa dice. Il primo confronta i classificatori; gli altri tre guardano più da vicino Random Forest, il classificatore di riferimento.

17. Noi contro il paper



Accuracy media di ogni classificatore, nostra e del paper.

Cosa c'è nel grafico

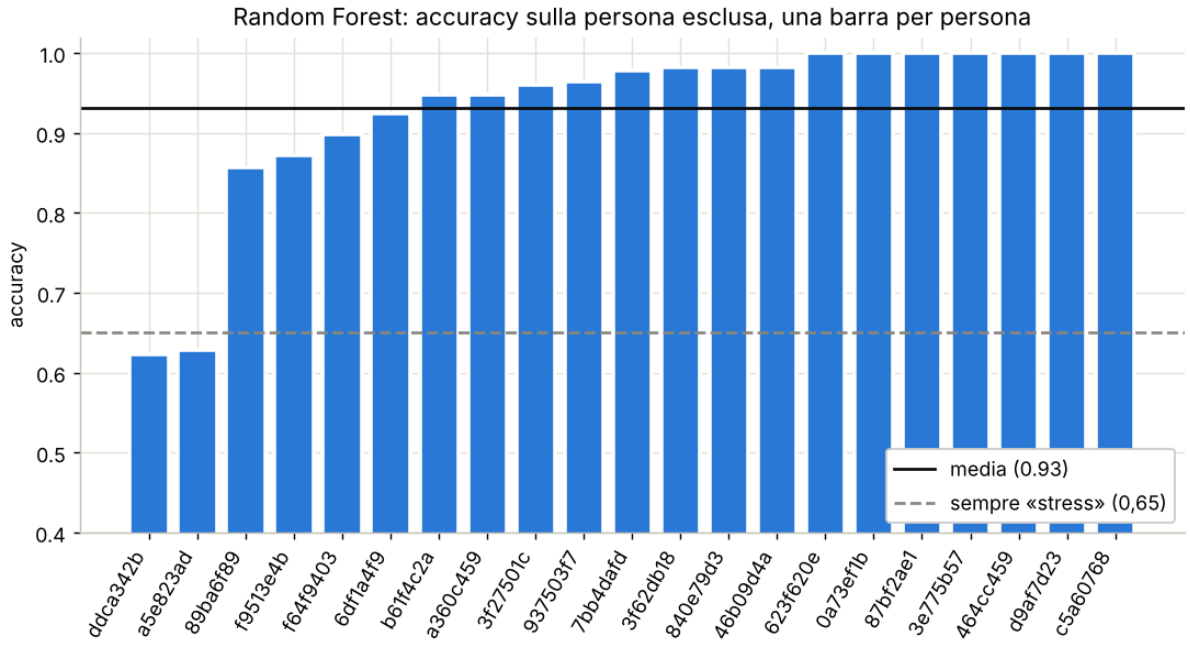
- **asse orizzontale**: i sei classificatori
- **asse verticale**: l'accuracy, cioè la percentuale di righe indovinate (1 = tutte giuste). Parte da 0,5 per far vedere meglio le differenze
- **barre blu**: i nostri risultati; **barre arancioni**: quelli della Tabella 4 del paper
- **linea tratteggiata grigia**: 0,65, l'accuracy di un modello che risponde sempre «stress» senza guardare i dati

Come si legge: per ogni classificatore si confrontano le due barre vicine. Poi si guarda quanto tutte stanno sopra la linea tratteggiata: più sono sopra, più il modello ha imparato davvero.

Cosa dice

- le barre blu sono **sopra o alla pari** di quelle arancioni per tutti e sei i classificatori: la replica è riuscita
- tutte le barre sono **ben sopra la linea del 65%**: i modelli hanno imparato qualcosa
- le barre si somigliano molto fra loro: per questi dati il tipo di classificatore conta poco

18. Accuracy per persona



Random Forest: accuracy sulla persona lasciata fuori, una barra per persona.

Cosa c'è nel grafico

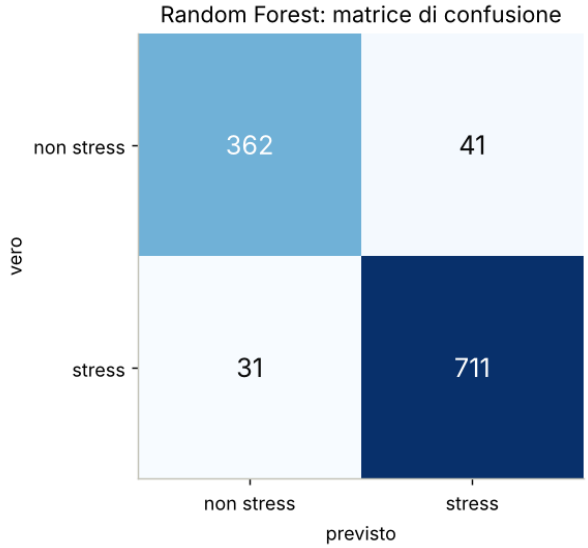
- **asse orizzontale**: le 21 persone, indicate con l'inizio del loro codice e **ordinate dalla peggiore alla migliore**
- **asse verticale**: l'accuracy ottenuta su quella persona quando era quella lasciata fuori
- **linea nera**: la media delle 21 barre (0,93), cioè il numero che finisce in tabella
- **linea tratteggiata grigia**: il 65% di chi risponde sempre «stress»

Come si legge: ogni barra risponde alla domanda «se questa persona fosse un utente nuovo, quanto funzionerebbe il braccialetto con lei?». Le barre a sinistra sono le persone su cui il modello va peggio.

Cosa dice

- per **19 persone su 21** l'accuracy supera 0,85, e per 7 è perfetta
- **due persone vanno molto peggio**, intorno a 0,62: ddca342b e a5e823ad
- a5e823ad è la persona senza riposo (escluso perché durava 82 minuti): ha solo righe di stress, e il modello ne scambia per riposo più di un terzo
- **la media nasconde queste differenze**: 0,93 sembra ottimo, ma su alcune persone il modello sbaglia molto. Spiega la deviazione standard alta (± 0,11). Anche il paper nota lo stesso fenomeno

19. Matrice di confusione



Random Forest: tutte le previsioni dei 21 giri insieme.

Cosa c'è nel grafico

- **righe**: la risposta vera (in alto «non stress», in basso «stress»)
- **colonne**: la risposta del modello (a sinistra «non stress», a destra «stress»)
- **numero in ogni casella**: quante righe della tabella sono finite lì
- **colore**: più scuro = più righe

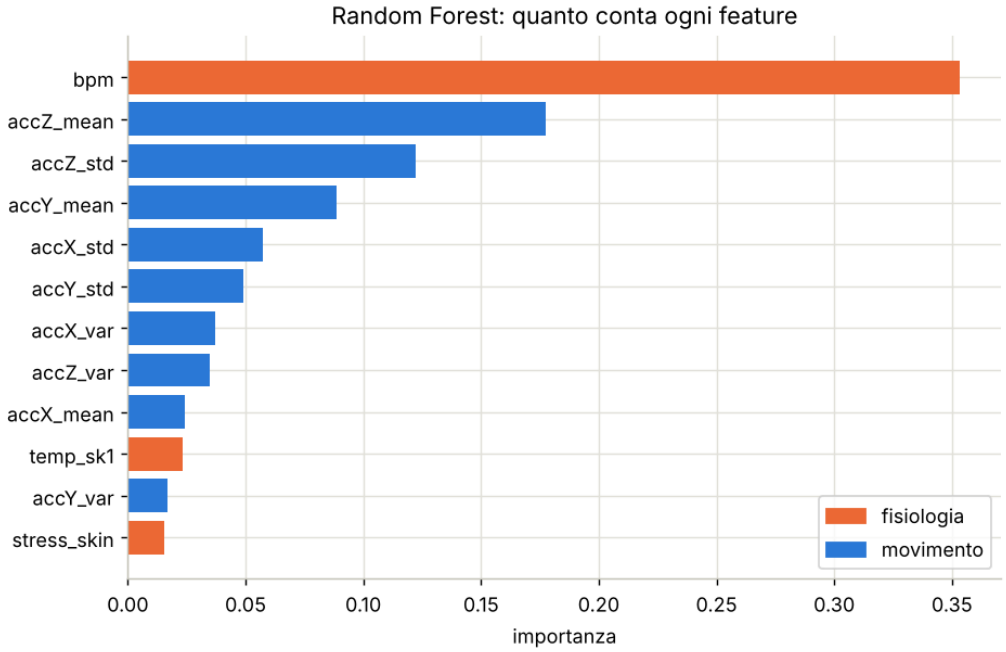
Come si legge: la **diagonale** (in alto a sinistra e in basso a destra) contiene le risposte giuste; le altre due caselle sono gli errori. Un buon modello ha numeri grandi sulla diagonale e piccoli fuori.

casella	numero	significato
in alto a sinistra	362	riposi riconosciuti
in basso a destra	711	stress riconosciuti
in alto a destra	41	**falsi allarmi**: riposo scambiato per stress
in basso a sinistra	31	**stress mancati**: lo stress c'era ma il modello non l'ha visto

Cosa dice

- gli errori sono pochi (72 su 1.145) e divisi abbastanza equamente fra falsi allarmi e stress mancati
- il modello riconosce il riposo nel 90% dei casi (362 su 403): non si limita a rispondere sempre «stress»
- nel paper, con kNN, i falsi allarmi sono 84 e gli stress mancati 31: noi abbiamo circa metà dei falsi allarmi
- in un sistema di sicurezza gli errori più gravi sono gli **stress mancati**

20. Importanza delle feature



Random Forest: quanto il modello usa ogni feature per decidere.

Cosa c'è nel grafico

- **asse verticale**: le 12 feature, dalla più usata (in alto) alla meno usata (in basso)
- **asse orizzontale**: l'importanza. Le 12 importanze sommano a 1, quindi 0,35 vuol dire «35% delle decisioni»
- **colore**: arancione = fisiologia (battito, sudore, temperatura); blu = movimento (accelerometro)

Come si legge: più la barra è lunga, più il modello si appoggia a quella feature. Il colore dice se si appoggia di più al corpo o al movimento.

Cosa dice

- la feature più usata è il **battito** (0,35)
- subito dopo c'è il **movimento**, soprattutto `accZ_mean` (0,18): la posizione del polso, la gravità vista nella Figura 4
- **sommate, le 9 feature di movimento pesano circa 0,61**, le 3 di fisiologia circa 0,39
- il **sudore** è l'ultima: coerente con la Figura 4, dove la standardizzazione appiattiva le differenze fra fasi

Il collegamento con il progetto: il modello si appoggia molto al movimento e alla posizione del polso. Non è per forza un errore, ma è da tenere presente quando si valutano le nuove feature.

21. Decisioni aggiunte

#	decisione	motivo
9	validazione leave-one-subject-out	come gli autori; misura su persone mai viste
10	iperparametri standard, senza grid search	il paper non dichiara i valori provati
11	F1 macro	come il paper; adatta a classi sbilanciate
12	random_state=42	risultati riproducibili